

09 · Capstone-Projekt & Architekturmuster

Edge, AUTOSAR und Capstone · EZS · DHBW Stuttgart

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 31

Das große Finale!

- Herzlich Willkommen zum 9. und letzten Termin unserer Vorlesung.
- Hinter uns liegen 8 Wochen intensiver Theorie und Praxis.
- Heute blicken wir auf das große Ganze:
 - Wie fügen sich unsere Micro-Konzepte in die Makro-Welt der Industrie ein?
 - Wie baut die Automobilindustrie Software (Exkurs AUTOSAR)?
- Am Nachmittag: Das **Capstone-Projekt** (Die Master-Prüfung für Ihr Board).

Notizen

Guten Morgen und herzlich willkommen zum großen Finale! Wir sind am neunten und letzten Termin unserer Vorlesung angekommen. Sie haben in den letzten Wochen wahnsinnig viel gelernt – von einfachen Interrupts bis hin zu Hardware-Schlössern im Multicore-System. Heute wollen wir den Kopf aus den Register-Handbüchern heben und auf das große Ganze schauen. Wir werfen einen Blick in die Industrie: Wie strukturiert man riesige Anlagen? Wie baut die Automobilindustrie aus 100 Steuergeräten ein funktionierendes Auto? Danach gehen wir gemeinsam ins Labor und Sie bearbeiten Ihr Capstone-Projekt. Sie werden heute Nachmittag ein System bauen, das alle Konzepte des Semesters vereint, und dieses System der BlueBox zur Timing-Analyse vorlegen.

Folie

2 / 31

Agenda für heute

- Semester-Rückblick: Die Reise vom C-Code zum RTOS.
- Reale Architekturmuster der Industrie (Edge, Redundanz).
- Safety-Muster (Fail-Safe vs. Fail-Operational & Lockstep).
- Exkurs: AUTOSAR (Der Automobil-Standard).
- Das Pflichtenheft für das Capstone-Labor.

Notizen

Was genau steht heute auf der Agenda? Wir starten mit einem kurzen Rückblick auf unsere Reise. Dann schauen wir uns an, wie moderne Cyber-Physische Systeme strukturiert werden. Wir besprechen Redundanzkonzepte und gehen nahtlos in einen Exkurs zu AUTOSAR über. Sie müssen als Embedded-Entwickler verstehen, wie die Automobilwelt tickt, auch wenn wir hier im Labor FreeRTOS verwenden. Den Abschluss des Vormittags bildet die Vorstellung Ihres Pflichtenhefts für das heutige Laborprojekt.

Folie

3/31

Die Reise dieses Semesters

1. **Woche 1–2:** Bare-Metal, Interrupts und das Latenz-Problem der Polling-Loop. Die Schedulability-Theorie (RMS).
2. **Woche 3–5:** Der Wechsel zum RTOS (FreeRTOS). Task-States, Queues und das *Deferred Interrupt Processing*.
3. **Woche 6–7:** Der zweite Kern erwacht. Shared Memory, Cache-Kohärenz (MPU) und Timing-Analysen mit der TASKING BlueBox.
4. **Woche 8:** Safety. Hardware Semaphores (HSEM) gegen Race Conditions und Watchdogs (WWDG) gegen den System-Crash.

Notizen

Lassen Sie uns kurz rekapitulieren, was Sie in diesem Semester alles erreicht haben. Wir haben ganz simpel mit Bare-Metal-Code angefangen und gemerkt, dass eine `while (1)`-Schleife für harte Deadlines nicht reicht. Wir haben die RMS-Schranke von Liu und Layland kennengelernt. Dann haben wir FreeRTOS integriert, gelernt, dass eine blockierte Task die beste Task ist, und Daten per Queue sicher transportiert. Danach wurde es anspruchsvoll: Wir haben den zweiten Prozessorkern geweckt, das Cache-Problem gelöst und gelernt, dass `printf` das Timing zerstört. Letzte Woche haben wir das System mit Hardware-Schlössern und Watchdogs bombensicher gemacht.

Folie

4/31

Erkenntnis 1: Determinismus schlägt Rohleistung

- In der IT-Welt (PC, Cloud) optimieren wir auf maximalen Durchsatz (Average Case).
- In der Echtzeit-Welt ist der **Worst-Case** das einzige, was zählt!
- **Die wichtigste Formel:** $R_i \leq D_i$ (Response Time $\leq$ Deadline).
- Wir opfern absichtlich CPU-Zyklen (OS-Overhead, Cache-Deaktivierung), um die Varianzen (Jitter) zu eliminieren.

Notizen

Was sind die Kernerkenntnisse, die Sie in Ihre berufliche Zukunft mitnehmen sollen? Erkenntnis Nummer 1: Determinismus schlägt Rohleistung. Wenn Sie später Software für einen Webserver schreiben, wollen Sie maximalen Durchsatz im Durchschnitt. In der Echtzeit interessiert uns der Durchschnitt nicht. Uns interessiert nur der absolut schlimmste Fall, die Worst-Case Execution Time. Wenn wir den Cache abschalten müssen, um Latenzen berechenbar zu machen, dann machen wir den Chip zwar langsamer, aber wir machen ihn sicher. Pünktlichkeit ist in unserer Branche wichtiger als Geschwindigkeit.

Folie

5/31

Erkenntnis 2: Synchronisation ist tödlich

- Die asynchrone Präemption ist ein fantastisches Werkzeug, um Tasks nach Priorität zu ordnen.
- **Aber sie zerreit unsere Datenstrukturen!**
- Die Gefahrenschwerpunkte:
 - *Race Conditions*: Read-Modify-Write wird unterbrochen.
 - *Deadlocks*: Zwei Tasks blockieren sich gegenseitig.
 - *Priority Inversion*: Eine unwichtige Task sperrt den Mutex und verdrängt die wichtigste Task im System (Mars Pathfinder).
- Die Lösung liegt immer im Architektur-Design, selten im reinen C-Code.

Notizen

Erkenntnis Nummer 2: Synchronisation ist tödlich, wenn sie falsch gemacht wird. Wir haben die Präemption eingeführt, um schnelle Reaktionszeiten zu garantieren. Doch diese Macht hat einen Preis. Wenn wir unsere kritischen Abschnitte nicht mit Mutexes oder HSEMs schützen, bekommen wir Heisenbugs und Datenkorruption. Wenn wir die Schlösser aber falsch setzen, stürzen wir in Deadlocks oder Prioritätsinversionen ab. Ein stabiles Echtzeitsystem entsteht nicht durch langes Tippen von C-Code, sondern durch sorgfältige Planung der Task-Architektur am Whiteboard.

Folie

6/31

Erkenntnis 3: Vertraue niemals der Software

- Selbst formell verifizierter Code kann durch Hardware-Einflüsse (EMI, Bit-Flips) crashen.
- Ein sicheres System verlässt sich **nie** nur auf die Logik.
- Wir brauchen Hardware-Fangnetze:
 - **MPU (Memory Protection Unit)**: Verhindert Stack Overflows physikalisch.
 - **IWDG / WWDG**: Starten das System hart neu, wenn Deadlocks auftreten oder das Timing aus dem Fenster fällt.

Notizen

Erkenntnis Nummer 3: Vertrauen Sie niemals Ihrer eigenen Software. Selbst wenn Sie den Code dreifach reviewen und statische Analysen darüber laufen lassen, kann ein einzelnes hochenergetisches Neutron aus dem Weltall Ihren Programm-Counter im RAM kippen. Das System wird crashen. Ein echtes Industrie-System fängt diese physikalischen Fehler durch Hardware-Redundanz ab. Die MPU stoppt falsche Speicherzugriffe in der Nanosekunde ihrer Entstehung, und unsere Hardware-Watchdogs fungieren als Totmannschalter.

Folie

7 / 31

Vom Mikrocontroller zum CPS

- Unsere Cortex-M-Platine ist heute selten ein isoliertes Gerät.
- Sie ist fast immer Teil eines **Cyber-Physical Systems (CPS)**.
- **CPS:** Die tiefe Verschmelzung von physischer Mechanik (Motor, Sensor) mit Software und globalen Netzwerken (Internet of Things).
- *Das führt zu extremen architektonischen Herausforderungen bezüglich Latenz und Bandbreite!*

Notizen

Lassen Sie uns nun den Blickwinkel öffnen. Bisher haben wir uns auf das beschränkt, was innerhalb unseres schwarzen Plastik-Chips passiert. Aber in der Realität sind moderne Echtzeitsysteme fast nie isoliert. Sie sind Knotenpunkte in einem Cyber-Physical System. Das bedeutet, wir haben auf der einen Seite rohe Physik – schwere Hydraulik-Pumpen oder chemische Sensoren. Und auf der anderen Seite haben wir eine Anbindung an globale Daten-Netzwerke, das Internet of Things. Das System muss also gleichzeitig einen Motor in Millisekunden regeln und kryptografisch verschlüsselte Netzwerk-Zertifikate parsen.

Folie

8 / 31

Das Edge-Computing Pattern

- „Senden wir einfach alle Sensordaten roh in die Cloud und berechnen die Regelung dort!“
- **Warum das fehlschlägt:**
 1. **Latenz:** Ping-Zeiten von 50 ms ins Internet töten jeden PID-Regler.
 2. **Bandbreite:** 40.000 Samples/Sek. mal 1000 Sensoren verstopfen das Netz.
 3. **Reliability:** Das WLAN fällt aus → Maschine crasht.
- **Die Lösung: Edge Computing.**
 - Die Daten werden „an der Kante“ (Edge) – also direkt auf dem Mikrocontroller – gefiltert und aggregiert.
 - Nur kompakte Metadaten („Temperatur OK“) wandern ins Netz.

Notizen

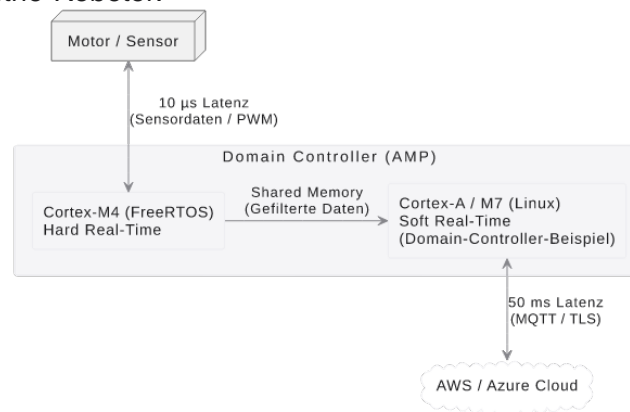
Ein riesiger Trend in diesem Umfeld ist das sogenannte Edge-Computing Pattern. Häufig kam aus der IT-Welt die Forderung: „Unsere Cloud-Server sind so schnell, lasst uns die billigen Mikrocontroller nur als dumme Datensammler nutzen. Wir schicken die Rohdaten einfach in die Cloud und regeln die Maschine von dort.“ Diese Idee ist in der Industrie spektakulär gescheitert. Warum? Wegen der Latenz. Wenn Ihr Sensor-Paket durch fünf Router läuft, hat es 50 Millisekunden Verzögerung. Damit können Sie keinen Roboterarm stabil regeln. Und wenn das WLAN abbricht, rammt der Arm ungebrems durch die Werkshalle. Die Lösung lautet Edge Computing: Wir erledigen die harte Echtzeit und Vorfilterung lokal auf dem Chip „an der Kante“ des Netzwerks. In die Cloud funken wir nur noch hoch verdichtete Metadaten.

Folie

9/31

Asymmetrische Lastverteilung in der Praxis

- Edge-Computing spiegelt exakt unsere AMP-Architektur aus Termin 6 wider!
- Ein realer Industrie-Roboter:



Notizen

Und genau hier schließt sich der Kreis zu unserer Multicore-Architektur. Das Edge-Computing Pattern ist der perfekte Anwendungsfall für Asymmetric Multiprocessing (AMP). Der Cortex-M4 hängt direkt an den Motoren. Er garantiert eine 10-Mikrosekunden-Latenz, liest Encoder aus und regelt hart in Echtzeit. Die Ergebnisse, die aufbereiteten Metadaten, schiebt er über das HSEM und den Shared Memory zum starken M7-Kern. Auf dem M7 läuft ein fettes Linux oder komplexe Bare-Metal-Kryptographie. Der M7 verpackt die Daten sicher per TLS-Zertifikat und funkt sie gemütlich über Ethernet in die Cloud. Die beiden Zeitdomänen sind physikalisch perfekt entkoppelt.

Safety-Muster: Fail-Safe vs. Fail-Operational

- Wenn ein kritischer Fehler auftritt (z. B. Watchdog Reset), wie soll das System reagieren?
- **Fail-Safe (Fehlersicher):**
 - Das System schaltet sich sofort in einen sicheren Zustand (Motor aus, Bremsen zu).
 - Beispiel: Industrieller Roboterarm, Eisenbahn-Signal, Waschmaschine.
- **Fail-Operational (Fehlertolerant):**
 - Das System darf nicht abschalten! Es muss mit reduzierter Leistung weiterarbeiten.
 - Beispiel: Autonomes Fahren auf der Autobahn (Level 4), Flugzeug-Fly-by-Wire.
 - Erfordert massive Hardware-Redundanz!

Notizen

Gehen wir noch einen Schritt weiter auf der Architekturebene. Wenn ein Fehler auftritt, wie reagiert unser System? In der Safety-Normung unterscheiden wir rigoros zwischen Fail-Safe und Fail-Operational. Fail-Safe ist einfach: Wenn der Watchdog Alarm schlägt, schalten wir den Motor stromlos. Der Roboterarm bleibt einfach stehen. Der Betrieb ist gestört, aber niemand wird verletzt. Fail-Operational ist die Meisterklasse. Wenn Sie bei 130 km/h im autonom fahrenden Auto sitzen und die CPU crasht, ist „Ausschalten“ keine gute Idee. Das System muss die Spur halten und geordnet auf den Standstreifen fahren. Das erfordert Architekturen, die den Ausfall ganzer Rechenzentren kompensieren können.

Redundanz-Architekturen

- Um Fail-Operational zu erreichen, vervielfachen wir die Hardware!
- **1oo2 (One-out-of-two):**
 - Zwei getrennte Kanäle berechnen dasselbe (Diagnose-/Vergleichsarchitektur).
 - Typisch: Beide Kanäle können bei Diskrepanz den **Safe State** erzwingen (Fail-Safe).
 - *Hot-Standby* (nahtlose Übernahme) ist eine mögliche Ausprägung — nicht die einzige Bedeutung von 1oo2.
- **2oo3 (Two-out-of-three):**
 - Drei unabhängige Controller (oft von verschiedenen Herstellern!).
 - Ein „Voter-Chip“ vergleicht die Ergebnisse.
 - Wenn CPU 1 sagt „Lenken“, aber CPU 2 und 3 sagen „Nicht lenken“, wird CPU 1 als defekt überstimmt! (Standard in der Luftfahrt).

Notizen

Um Fail-Operational oder zumindest robustes Fail-Safe zu erreichen, brauchen wir Redundanz. 1oo2 bedeutet „One out of two“: zwei Kanäle. In vielen Safety-Architekturen können beide Kanäle bei einem

Fehler den Safe State erzwingen — das ist Diagnose und Absicherung, nicht zwingend Hot-Standby. Hot-Standby (A aktiv, B als Schatten, nahtlose Übernahme) ist nur eine mögliche Ausprägung. In extrem kritischen Bereichen, wie der Flugsteuerung eines Airbus, nutzt man oft 2oo3: drei Computer, ein Voter. Bei einem Bit-Flip überstimmen die zwei gesunden Chips den defekten. Oft kommen Chips unterschiedlicher Hersteller zum Einsatz (Diversität), damit ein systematischer Compiler-Bug nicht alle drei gleichzeitig trifft.

Folie

12/31

Lockstep-Prozessoren (Hardware-Safety)

- Wie erkennt ein Controller, dass er selbst gerade einen Bit-Flip hatte?
- Die Lösung der Automobilindustrie: **Lockstep-Architektur**.
- Ein spezieller Chip (z. B. Infineon AURIX, TI SafeTI).
- In einem Silizium-Gehäuse stecken zwei identische Kerne.
- Beide bekommen exakt denselben Takt und exakt dieselben Befehle.
- Ihre Ausgänge werden in der Hardware Bit für Bit verglichen.
- Ist nur ein einziges Bit unterschiedlich → Sofortiger Hardware-Halt!

Notizen

Die Automobilindustrie hat für Bremsen und Lenkung noch einen anderen Ansatz perfektioniert: Den Lockstep-Prozessor. Das ist Hardware-Paranoia in Perfektion. Sie kaufen einen Chip, auf dem zwei völlig identische CPU-Kerne sitzen. Sie als Programmierer sehen aber nur einen. Sie schreiben ein C-Programm. Die Hardware füttert jeden Befehl an beide Kerne, die im absoluten Gleichschritt (Lockstep) rechnen. Kurz bevor das Signal auf den Bus geht, vergleicht ein Hardware-Gatter die Ergebnisse beider Kerne. Wenn Core A eine 1 ausgeben will und Core B eine 0, weiß die Hardware: Jemand von uns beiden ist defekt. Der Chip geht im Bruchteil einer Nanosekunde in den sicheren Fehlerzustand.

Folie

13/31

Zusammenfassung Architekturmuster

- Edge-Computing verlagert harte Echtzeit aus der Cloud zurück auf den Mikrocontroller.
- Moderne Systeme trennen weiche und harte Echtzeit oft asymmetrisch (AMP).
- Fail-Safe Systeme schalten bei einem Fehler sicher ab (Watchdog Reset).
- Fail-Operational Systeme erfordern Redundanz (1oo2, 2oo3) oder Lockstep-Chips, um den Betrieb aufrechtzuerhalten.

Notizen

Fassen wir diesen Exkurs in die Industrie-Architekturen zusammen. Sie sehen, dass unsere Multicore-Programmierung aus der Vorlesung direkt in das moderne Edge-Computing-Muster einzufließt. Wenn Sie in die Luftfahrt oder ins autonome Fahren gehen, müssen Sie über Ihre C-Schleifen hinaus denken und

Fail-Operational-Muster entwerfen. Einer der Treiber für standardisierte Software-Strukturen, um diese gewaltige Komplexität beherrschbar zu machen, kommt aus der deutschen Automobilindustrie.

Folie

14/31

Exkurs AUTOSAR

- AUTOSAR = AUTomotive Open System ARchitecture.
- Warum schauen wir uns das an?
- Jeder Embedded-Entwickler in Europa wird früher oder später mit diesem Begriff konfrontiert.
- Es löst Probleme, auf die wir in FreeRTOS stoßen würden, wenn unser Projekt auf 100 Entwickler anwächst!

Notizen

Und das bringt uns zum letzten Theorie-Exkurs für dieses Semester: AUTOSAR. Auch wenn wir im Labor das quelloffene FreeRTOS nutzen, möchte ich Sie nicht ohne Grundkenntnisse in AUTOSAR ins Berufsleben entlassen. Fast die gesamte europäische Automobil-Zulieferer-Industrie spricht diese Sprache. AUTOSAR ist kein spezieller Code, es ist ein massiver Standard, der erfunden wurde, um Projekte mit hunderten von Softwareentwicklern überhaupt noch kontrollieren zu können.

Folie

15/31

Die Automobilkrise (Anfang der 2000er)

- Das Problem:
 - Ein modernes Auto hat bis zu 100 Steuergeräte (ECUs).
 - Jedes Steuergerät wurde von einem anderen Zulieferer programmiert (Bosch, Continental, Hella).
 - Jeder nutzte seine eigenen Treiber, sein eigenes OS und seine eigenen Datenformate.
- Wenn BMW einen Brems-Algorithmus von Bosch auf ein Steuergerät von Continental portieren wollte → Alles musste von Hand neu geschrieben werden!
- Der „Spaghetti-Code“ skalierte nicht mehr mit der rasanten Zunahme an Software im Auto.

Notizen

Warum hat man AUTOSAR überhaupt erfunden? Gehen wir zurück in die frühen 2000er Jahre. Autos wurden plötzlich fahrende Computer. Ein Premium-Fahrzeug hatte bald 100 kleine Steuergeräte. Der OEM, also Mercedes oder VW, bestellte das Motorsteuergerät bei Bosch, den Bremsregler bei Conti und das Radio bei Alpine. Jeder Zulieferer lötete seinen eigenen Lieblings-Chip auf die Platine, schrieb seine eigenen Hardware-Treiber und wickelte seinen C-Code untrennbar um diese Treiber. Wenn der OEM den grandiosen Brems-Algorithmus von Bosch plötzlich auf der billigeren Platine von Continental laufen

lassen wollte, war das ein Projekt für drei Jahre, weil jede Codezeile neu geschrieben werden musste. Die Software-Kosten explodierten.

Folie

16/31

Die Geburt von AUTOSAR

- 2003: BMW, Daimler, VW, Bosch, Continental (und später viele andere) schließen sich zusammen.
- Ihr gemeinsames Motto:
 - „Cooperate on standards, compete on implementation.“
 - (Kooperiert bei den Standards, konkurriert bei der Umsetzung).
- Das Ziel:
 - Eine strikte Standardisierung der Software-Architektur.
 - Hardware-Treiber (z. B. für STM32) und Applikations-Code (z. B. ABS-Algorithmus) müssen vollständig voneinander getrennt sein.

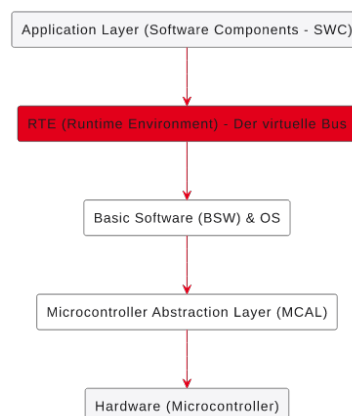
Notizen

Das war der Punkt, an dem sich die erbitterten Konkurrenten der Industrie an einen Tisch setzen mussten. BMW, Daimler, VW, Bosch und Conti gründeten 2003 das AUTOSAR-Konsortium. Ihr Leitsatz war genial: „Cooperate on standards, compete on implementation“. Das heißt: Lasst uns nicht mehr darüber streiten, wie eine C-Funktion für einen CAN-Bus-Treiber auszusehen hat. Wir definieren einen eisernen Standard. Konkurrieren tun wir in Zukunft nur noch auf der Ebene der Algorithmen. Das Ziel war klar: Die physikalische Hardware und der logische C-Code der Applikation müssen vollständig voneinander entkoppelt werden.

Folie

17/31

Das AUTOSAR Schichtenmodell



- Die Architektur wird in streng getrennte, waagerechte Ebenen zerschnitten.

Notizen

Das Herzstück dieses Standards ist das AUTOSAR Schichtenmodell. Anstatt den Code als wilden Haufen C-Dateien zu schreiben, wird das System streng horizontal in Layer zerschnitten. Ganz unten liegt die Hardware. Darüber liegt der MCAL, der die Register abstrahiert. Dann kommt die gewaltige Basic Software Schicht, in der das OS und die Netzwerkprotokolle liegen. Die magische rote Linie ist die RTE, die Runtime Environment. Und ganz oben, völlig isoliert vom Rest der Welt, schweben die Software Components, unsere Applikationen. Wir schauen uns diese Schichten kurz von unten nach oben an.

Folie

18/31

Schicht 1: MCAL & Basic Software (BSW)

- **MCAL (Microcontroller Abstraction Layer):**
 - Hier liegen die direkten Hardware-Treiber (z. B. SPI, ADC, PWM).
 - Dieser Code ist das Einzige, was sich ändert, wenn man von Infineon zu NXP wechselt!
- **Basic Software (BSW):**
 - Das Betriebssystem (AUTOSAR OS), CAN/Ethernet-Kommunikations-Stacks, Diagnose-Speicher, Memory-Verwaltung.
 - Hochgradig standardisiert. Hunderte vordefinierte C-Schnittstellen.

Notizen

Ganz unten auf dem blanken Silizium liegt der MCAL. Das ist im Prinzip das, was wir bisher als STM32-HAL bezeichnet haben. Der MCAL verbirgt die spezifischen Hardware-Register vor dem restlichen Code. Wenn BMW beschließt, im nächsten Modellzyklus von NXP-Chips auf Infineon-Chips zu wechseln, wird nur diese eine, unterste Schicht ausgetauscht. Darüber liegt die riesige BSW. Sie beinhaltet das Betriebssystem, komplexe CAN-Bus-Stacks, den NVRAM für Fehlercodes. Dieser Block ist so stark genormt, dass Zulieferer wie Vector Informatik oder Elektrobit diese Basic Software fertig als Produkt an die Autohersteller verkaufen können.

Das AUTOSAR OS (im Vergleich zu FreeRTOS)

- Das AUTOSAR Betriebssystem basiert auf dem älteren OSEK/VDX Standard.
- Es gibt einen gewaltigen Unterschied zu FreeRTOS:
- **Statische Konfiguration!**
- In FreeRTOS können wir zur Laufzeit mit `osThreadNew()` wild neue Tasks anlegen.
- In AUTOSAR ist das streng verboten!
- Alle Tasks, Prioritäten, Alarmer und Queues müssen vorab in XML/ARXML-Dateien definiert werden.
- Der Code-Generator baut daraus ein absolut fixes System.
- Vorteil: Mathematisch 100% nachweisbares Worst-Case-Verhalten (Keine Out-of-Memory Fehler zur Laufzeit!).

Notizen

Das Betriebssystem im Herzen der BSW ist besonders spannend für uns. Es unterscheidet sich massiv von unserem FreeRTOS. In FreeRTOS rufen wir `osThreadNew` auf und das OS sucht sich zur Laufzeit dynamisch ein bisschen RAM für den TCB zusammen. In AUTOSAR ist jede Art von dynamischer Speicherallokation im laufenden Betrieb verboten! Das OS ist rein statisch. Wenn Sie eine Task wollen, müssen Sie diese Wochen vorher in einem komplexen XML-Dokument am PC definieren. Ein Konfigurator generiert daraus starren C-Code. Sie können zur Laufzeit keine Task erschaffen oder löschen. Das klingt mühsam, ist für die harte Safety aber überlebenswichtig: Es kann niemals zu „Out of Memory“-Abstürzen kommen, und die Latenz-Zeiten sind zu 100% formal beweisbar.

Schicht 2: Die RTE (Runtime Environment)

- Das Meisterstück von AUTOSAR.
- Die RTE ist ein virtueller Bus.
- Wenn Software-Komponente A mit B reden will, ruft sie nicht C-Funktionen auf! Sie ruft die RTE auf.
- Die RTE routet die Daten:
 - **Fall 1:** Komponente B ist auf demselben Chip → RTE legt Daten in RAM (wie eine Queue).
 - **Fall 2:** Komponente B ist auf einem anderen Steuergerät im Heck des Autos → RTE packt die Daten auf den CAN-Bus!
- Für Komponente A sieht der C-Code in beiden Fällen exakt gleich aus!

Notizen

Über der Basic Software liegt das Meisterstück der Architektur: Die Runtime Environment, kurz RTE. Das ist kein C-Code, den ein Mensch schreibt. Die RTE wird generiert und fungiert als virtueller Bus (VFB). Wenn Ihre ABS-Software den aktuellen Radsensor-Wert lesen will, fragt sie die RTE. Der geniale Trick: Dem ABS-Algorithmus ist es völlig egal, ob der Code für den Sensor auf demselben Mikrocontroller mitläuft, oder ob der Sensorwert von einem Steuergerät am anderen Ende des Autos über den CAN-Bus kommt. Die RTE abstrahiert die gesamte Netzwerk-Infrastruktur weg. Sie nimmt den Funktionsaufruf entgegen und entscheidet intern, ob sie ihn in einen RAM-Puffer legt oder in ein CAN-Bus-Paket schnürt.

Folie

21 / 31

Schicht 3: Application Layer (SWC)

- Hier liegen die Software Components (SWC).
- Das ist der echte Mehrwert (z. B. der Regelalgorithmus für den Motor).
- Der Paradigmenwechsel:
 - Der Programmierer dieser Schicht hat keine Ahnung, wie der Mikrocontroller heißt, wie viel MHz er hat oder wo die Register liegen.
 - Der C-Code greift ausschließlich auf die abstrakten RTE-Funktionen zu.
- Dadurch wird der Code 100% wiederverwendbar und portabel über Fahrzeug-Generationen hinweg!

Notizen

Ganz oben, abgehoben von allen Hardware-Problemen, liegt der Application Layer mit den Software-Komponenten (SWCs). Hier programmieren die Regelungstechniker ihre Algorithmen. Für einen Entwickler in dieser Schicht existieren keine Timer-Register, keine Interrupts und kein SysTick. Er programmiert nur noch gegen saubere Schnittstellen. Durch diese vollständige Isolation der Logik erreicht die Automobilindustrie das Ziel von 2003: Sie können eine SWC für die Geschwindigkeitsregelung heute in einem VW Golf testen und morgen auf einen völlig anderen Chip-Hersteller für einen Audi übertragen. Sie müssen nicht eine Zeile C-Code anpassen.

Ausblick: AUTOSAR Adaptive

- Die Classic Platform (C-Code, harte Echtzeit) dominiert Mikrocontroller.
- Aber: Autonomes Fahren (KI, Kameras) braucht Rechenzentren auf Rädern!
- **AUTOSAR Adaptive Platform:**
 - Für Hochleistungs-Chips (Cortex-A Kerne, GPUs).
 - Basiert nicht mehr auf OSEK, sondern auf POSIX-Betriebssystemen (Linux / QNX).
 - Code wird oft in modernem C++ (Objektorientiert) geschrieben.
 - Ermöglicht dynamische Updates Over-the-Air (OTA).

Notizen

Kleine Ergänzung: Alles, was wir gerade besprochen haben, nennt sich die „AUTOSAR Classic Platform“ und dominiert die Cortex-M Mikrocontroller. Aber mit der Einführung von autonomen Fahren, wo wir neuronale Netze und gigantische Kameradatenströme verarbeiten müssen, stößt die Classic Platform an ihre Grenzen. Dafür hat das Konsortium die „Adaptive Platform“ geschaffen. Sie läuft auf Cortex-A Hochleistungs-Chips, nutzt als Basis echte POSIX-Systeme wie Linux oder QNX und programmiert wird fast nur noch in modernem, objektorientiertem C++. Es ist quasi das Bindeglied zwischen klassischer IT und dem Steuergerät und erlaubt unter anderem saubere Software-Updates „Over-The-Air“.

Warum AUTOSAR nicht immer die Lösung ist

- Wenn AUTOSAR so genial ist, warum nutzen wir im Labor **FreeRTOS**?
- **Riesiger Tool-Overhead:**
 - AUTOSAR programmiert man nicht in einem einfachen Text-Editor. Man benötigt gigantische Konfigurations-Werkzeuge (z. B. von Vector oder Elektrobit).
 - Lizenzen für diese Tools kosten oft zehntausende Euro pro Arbeitsplatz.
- **Steile Lernkurve:**
 - Ein einfaches „Blink-LED“-Projekt in AUTOSAR aufzusetzen, kann für einen Anfänger Wochen dauern (hunderttausende Zeilen generierter Code).
- **Fazit:** Für komplexe Automotive-Netzwerke unverzichtbar. Für smarte Sensoren, IoT-Devices oder schnelle Prototypen ist FreeRTOS die deutlich agilere und bessere Wahl!

Notizen

Bei all dem Lob für AUTOSAR müssen wir realistisch bleiben. Warum lehre ich Ihnen in diesem Kurs FreeRTOS und nicht AUTOSAR? Weil AUTOSAR für kleinere Projekte der absolute Overkill ist. Um ein AUTOSAR-Projekt zu starten, brauchen Sie sündhaft teure Software-Werkzeuge. Sie müssen hunderte Parameter in XML-Dateien konfigurieren, bevor der Code-Generator auch nur eine einzige C-Datei

ausspuckt. Ein „Blink-LED“-Programm dauert Wochen. Für IoT-Geräte, smarte Kaffeemaschinen oder Industrie-Sensoren ist AUTOSAR völlig überdimensioniert. Hier glänzt FreeRTOS mit seinem winzigen Footprint, der einfachen C-API und seiner Agilität.

Folie

24 / 31

Fazit AUTOSAR

- Auch wenn Sie im Job FreeRTOS programmieren, hilft Ihnen das Wissen über AUTOSAR massiv!
- **Architektur-Disziplin:**
 - Trennen Sie in Ihrem C-Code immer strikt die Hardware-Treiber (MCAL-Gedanke) von der Geschäftslogik (SWC-Gedanke).
 - Wenn Sie nächstes Jahr von STM32 auf einen ESP32 wechseln, müssen Sie dann nur eine dünne Treiberschicht neu schreiben!
- **Statische Allokation:**
 - Gewöhnen Sie sich an, auch in FreeRTOS Tasks und Queues *statisch* anzulegen (statt dynamischem Heap über `osThreadNew/malloc`, z. B. mit `xTaskCreateStatic`), um Out-of-Memory-Abstürze zur Laufzeit zu eliminieren.

Notizen

Das Fazit dieses Exkurses: Auch wenn Sie niemals in der Automobilindustrie arbeiten, adaptieren Sie die Denkmuster von AUTOSAR! Wenn Sie Ihren FreeRTOS-Code schreiben, vermischen Sie keine I2C-Register-Befehle tief in Ihren Filter-Algorithmen. Bauen Sie sich gedanklich eine eigene kleine RTE. Trennen Sie Treiber und Applikation. Und übernehmen Sie die Paranoia von AUTOSAR bezüglich Speicher: Vermeiden Sie dynamisches `malloc` zur Laufzeit, wo es nur geht. Legen Sie Ihre Puffer statisch im Vorfeld an. Das macht Ihr System sofort um ein Vielfaches robuster.

Folie

25 / 31

Das Capstone-Projekt

- Wir verlassen die Theorie und kommen zum praktischen Höhepunkt des Semesters.
- **Was ist ein Capstone-Projekt?** (Aus dem Englischen: Der Schlussstein eines Gewölbes).
- **Ihr Ziel heute:**
 - Sie programmieren kein isoliertes Feature mehr.
 - Sie bauen ein komplettes, asymmetrisches Multicore-System.
 - Das System ist gegen Hardware- und Softwarefehler abgesichert.
 - Sie beweisen das Timing am Ende mit der **TASKING BlueBox**.

Notizen

Damit schließen wir das Buch der Architekturtheorie und kommen zu Ihrem Meisterstück. Das heutige Labor ist das Capstone-Projekt. Der Capstone, der Schlussstein, ist der Stein, der am Ende in einen gemauerten Torbogen gesetzt wird und das gesamte Konstrukt trägt und zusammenhält. Genau das machen Sie heute. Sie fügen alle Puzzleteile der letzten 8 Wochen – Queues, Mutexe, Interrupts, Multicore, Watchdogs und Tracing – zu einer einzigen, massiven Architektur zusammen.

Folie

26 / 31

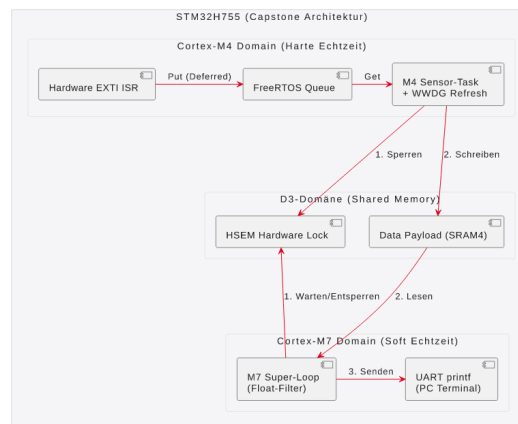
Das Szenario: Der Smart Industrie-Sensor

- **Die fiktive Kundenanforderung:**
 - Ein industrieller Sensor liefert zeitkritische Hardware-Impulse (Button/EXTI).
 - Die Impulse müssen auf die Mikrosekunde genau erfasst werden (harte Echtzeit).
 - Die Sensordaten müssen aufwändig gefiltert werden (Float-Berechnungen, weiche Echtzeit).
 - Das System darf bei Code-Hängern niemals für immer einfrieren (Safety).
 - Das System muss am PC Log-Daten ausgeben.
- **Ihre Waffe:** Der Dual-Core STM32H755.

Notizen

Das Szenario für unser Labor simuliert einen realen Industrie-Auftrag. Ihr Kunde braucht einen smarten Sensor. Die Hardware liefert schnelle, unregelmäßige Interrupts. Diese müssen hart in Echtzeit abkassiert werden, da wir sonst Impulse verlieren. Danach muss das Rauschen der Impulse mit komplexer Fließkomma-Mathematik gefiltert und über eine langsame UART-Schnittstelle an einen Leitrechner gesendet werden. Das Ganze muss absolut absturzsicher sein. Das perfekte Szenario für unseren Dual-Core Chip.

System-Architektur (Gesamtübersicht)



- **M4 (FreeRTOS):** Deferred Interrupt Processing → Queue → Sensor-Task mit WWDG.
- **Shared Memory (SRAM4):** Payload unter HSEM-Schutz.
- **M7 (Bare-Metal):** Float-Filter und UART-Ausgabe an den PC.
- *Wichtig:* `printf`/UART bewusst nur auf dem Soft-Echtzeit-Kern (M7) — nicht im harten M4-Pfad (vgl. Termin 7).

Notizen

Die Gesamtarchitektur Ihres Capstone-Systems sieht so aus: Auf dem Cortex-M4 läuft FreeRTOS. Der Hardware-Interrupt (EXTI) schreibt nur knapp in eine Queue – Deferred Interrupt Processing aus Woche 5. Die Sensor-Task holt die Events, sperrt das HSEM und schreibt den Payload in den SRAM4. Auf dem Cortex-M7 läuft Bare-Metal: Er wartet auf bzw. nutzt denselben Hardware-Lock, liest die Daten, rechnet den Float-Filter und schickt Logs über UART. Der Window Watchdog auf dem M4 überwacht, dass die harte Echtzeit-Task im erlaubten Zeitfenster bleibt. Alle Bausteine der Vorlesung greifen hier ineinander.

Capstone-Pflichtenheft (Muss-Kriterien)

- Verdichtet aus Lab 9 — das System **muss**:
 1. **EXTI** → **Queue (M4)**: Button PC13 als EXTI; ISR nur Deferred Put (`timeout = 0`).
 2. **HSEM-Sync**: SRAM4-Payload (`sd`) nur unter `HSEM_0` lesen/schreiben (beide Kerne).
 3. **WWDG**: Window + EWI auf dem M4; Refresh nur im Allowed Window (auch ohne Button-Event).
 4. **M7**: Float-Filter + `UART_printf` ausschließlich auf dem Soft-Echtzeit-Kern.
 5. **BlueBox**: E2E-Latenz (EXTI → HSEM-Release) und Jitter der `M4_SensorTask` messen.
- **MPU**: Shared Region Non-Cacheable (wie Termin 6).

Notizen

Das Pflichtenheft verdichtet Übungsblatt 9: harte Interrupt-Pipeline auf dem M4, asynchrone Multicore-Übergabe per HSEM, Soft-Pfad mit Filter und UART auf dem M7, Speicherschutz wie in Woche 6 — und der BlueBox-Beweis am Ende.

Abnahmekriterien (Definition of Done)

- **Funktion**: Tastendruck erscheint als gefilterter M7-UART-Log; kein Deadlock/Datenverlust unter HSEM.
- **Safety**: WWDG hält im Normalbetrieb; Early-/Late-Test zeigt Reset bzw. EWI-Verhalten.
- **Timing (BlueBox)**: E2E-Latenz im niedrigen μs -Bereich und stabil; M4-Jitter ≈ 0 trotz M7-Last/UART.
- **Architektur**: `printf`/UART nur auf M7; M4-ISR bleibt kurz.

Notizen

Abnahme heißt: Das System tut, was der Kunde will, und Sie können es messen. Funktion ohne Crash, Watchdog greift im Fehlerfall, BlueBox zeigt kurze stabile Latenz und praktisch keinen M4-Jitter.

Labor-Checkliste Capstone

1. CubeMX: Dual-Core, FreeRTOS nur CM4, EXTI, WWDG+EWI, HSEM, Queue + High-Prio-Task
2. Shared Memory `sd` + Linker `RAM_D3 / NOLOAD` + MPU Non-Cacheable
3. Deferred Interrupt Processing (ISR kurz, Arbeit in der Task)
4. Crash-Tests Early/Late (WWDG) und kurzer EWI-Blink (LD3 / PB14)
5. BlueBox: End-to-End-Latenz und Jitter der M4-Pipeline verifizieren
 - Teilabnahme möglich: zuerst nur M4-Pfad, dann Multicore, dann Trace

Notizen

Arbeiten Sie die Checkliste der Reihe nach ab. Wenn die Zeit knapp wird: Minimum Viable ist M4 mit Queue+WWDG und nachweisbarem Refresh im Fenster; HSEM/M7 und BlueBox als Erweiterung.

Die Essenz des Capstone

- Sie bauen heute kein Übungsfragment — Sie bauen eine **Serien-Architektur**.
- Alle Semesterbausteine greifen ineinander: ITC, Deferred IRQ, AMP, HSEM, WWDG, MPU, Trace.
- Erfolg heißt: funktionale Korrektheit *und* nachweisbares Timing.
- Viel Erfolg im Labor — und in der Klausur!

Notizen

Das Capstone ist der Schlussstein. Wenn Ihr System die Impulse jitterfrei erfasst, den Soft-Pfad entkoppelt und den Watchdog im Fenster hält, haben Sie die Lernziele der Veranstaltung erreicht. Viel Erfolg!